

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE: CSA06COURSE OUTCOME COVERED

NAME: A SHIVA SHANKAR REDDY

REG NO:192425174

CO 4: Solve optimization problems using dynamic programming and greedy strategies.

(BL4, BL5)

Annexure A

Industry Problem Solving Task

1. Problem Statement

Given a set of three items with weights [10, 20, 30] and corresponding values [60, 100, 120], and a knapsack (or resource) capacity of 50, the task is to solve the item-selection problem using two different algorithmic strategies: the 0/1 Knapsack Problem, solved using Dynamic Programming (DP), where each item must be taken entirely or not at all; and the Fractional Knapsack Problem, solved using a Greedy strategy, where items may be split and a fraction of an item's weight and value can be taken. The two solutions are then compared to analyze differences in optimality, the strengths and limitations of each method, and to derive insight into when each approach should be selected in practice.

Given dataset:

Item	Weight	Value	Value/Weight Ratio
Item 1	10	60	6.0
Item 2	20	100	5.0
Item 3	30	120	4.0

Knapsack / resource capacity: 50 units.

2. Solving via 0/1 Knapsack – Dynamic Programming

In the 0/1 Knapsack version, each item must either be fully included or fully excluded. The DP recurrence used is:

```
dp[i][w] = dp[i-1][w]                                if weight[i] > w
dp[i][w] = max( dp[i-1][w],
                value[i] + dp[i-1][w - weight[i]] )    if weight[i] <= w
```

Evaluating all feasible combinations of the three items against the capacity of 50 gives the following possibilities:

Combination	Total Weight	Total Value
Item1 only	10	60
Item2 only	20	100
Item3 only	30	120
Item1 + Item2	30	160
Item1 + Item3	40	180
Item2 + Item3	50	220
Item1 + Item2 + Item3	60 (exceeds capacity – infeasible)	—

The best feasible combination within the capacity of 50 is Item 2 + Item 3, using exactly $20 + 30 = 50$ units of capacity and yielding a total value of $100 + 120 = 220$. This is confirmed by the DP table computation ($dp[3][50] = 220$), and is the optimal solution for the 0/1 Knapsack version of this problem.

2.1 0/1 Knapsack – DP Result

Selected items : Item2, Item3
Total weight : 50 / 50
Maximum value : 220

3. Solving via Fractional Knapsack – Greedy Strategy

In the Fractional Knapsack version, items may be split, so the Greedy strategy sorts items by value-to-weight ratio in descending order and fills the knapsack by taking as much of the highest-ratio item as possible, moving to the next item once the current one is exhausted or the capacity is filled.

Ratios (value/weight): Item 1 = $60/10 = 6.0$, Item 2 = $100/20 = 5.0$, Item 3 = $120/30 = 4.0$. Sorted in descending order of ratio: Item 1, Item 2, Item 3 (this happens to already be the given order).

3.1 Greedy Selection Walkthrough

Item Taken	Ratio	Weight Taken	Value Gained	Remaining Capacity
Item 1	6.0	10 (100%)	60	40
Item 2	5.0	20 (100%)	100	20
Item 3	4.0	20 of 30 (66.7%)	80	0

Item 1 is taken fully (weight 10, value 60), leaving 40 units of capacity. Item 2 is taken fully (weight 20, value 100), leaving 20 units of capacity. Item 3 cannot be taken fully (it requires 30 units but only 20 remain), so $20/30 = 0.667$ of Item 3 is taken, contributing $0.667 \times 120 = 80$ to the total value. The knapsack is now exactly full ($10 + 20 + 20 = 50$).

3.2 Fractional Knapsack – Greedy Result

Selected items : Item1 (100%), Item2 (100%), Item3 (66.7%)
Total weight : 50 / 50
Maximum value : $60 + 100 + 80 = 240$

4. Comparison of the Two Solutions

Aspect	0/1 Knapsack (DP)	Fractional Knapsack (Greedy)
Items selected	Item 2 (full), Item 3 (full)	Item 1 (full), Item 2 (full), Item 3 (66.7%)
Total weight used	50	50
Total value obtained	220	240
Item divisibility	Not allowed (all-or-nothing)	Allowed (fractions permitted)
Algorithm used	Dynamic Programming (exhaustive, tabulated)	Greedy (ratio-based, single pass after sorting)
Optimality guarantee	Guaranteed optimal for the 0/1 constraint	Guaranteed optimal for the fractional constraint

The Fractional Knapsack (Greedy) solution achieves a higher total value (240) than the 0/1 Knapsack (DP) solution (220). This does not mean Greedy is a “better” algorithm in general – it means the two solutions are answering two different problems with different constraints. Allowing Item 3 to be split lets the greedy method extract additional value (80) from the remaining 20 units of capacity that the 0/1 version is forced to leave partially unused in any combination not equal to Item2+Item3. The Fractional Knapsack value obtained by Greedy is, in fact, a valid upper bound on the 0/1 Knapsack optimal value for the same instance, which is consistent with the result here ($240 \geq 220$).

5. Strengths and Limitations of Each Method

5.1 Dynamic Programming (0/1 Knapsack)

- Strength: Always produces the true optimal solution when items cannot be divided, because it systematically evaluates every relevant combination via overlapping subproblems and optimal substructure.
- Strength: Directly models real-world all-or-nothing decisions (e.g., hosting a whole application, accepting a whole order, selecting an entire project).
- Limitation: Higher computational cost – $O(n \times W)$ time and space, which grows with capacity W . For very large capacities or item counts, this becomes expensive (pseudo-polynomial, not polynomial in input size).
- Limitation: Requires building and storing a table, which is more complex to implement than a greedy pass.

5.2 Greedy (Fractional Knapsack)

- Strength: Very efficient – $O(n \log n)$ time, dominated by sorting items by ratio; no table required.
- Strength: Simple to implement and reason about; produces the optimal solution whenever items are genuinely divisible.
- Limitation: Only optimal when fractional selection is a valid real-world option. When items are indivisible, Greedy can produce a solution that is infeasible (if applied naively without capping fractions) or suboptimal (if restricted to whole items only, as shown by the mismatch with the DP result here).
- Limitation: Greedy offers no optimality guarantee for the 0/1 variant of the problem; it optimizes locally (by ratio) rather than evaluating the global combination space.

6. When Greedy Provides Optimal Results vs. When DP Is Necessary

6.1 Greedy is Optimal When:

- Items or resources can genuinely be divided or partially allocated (e.g., liquid bulk cargo, bandwidth, CPU-time slices, money/currency amounts).
- The problem has the greedy-choice property: a locally optimal choice (highest ratio first) combined with optimal solutions to the remaining subproblem leads to a global optimum – true for Fractional Knapsack, Activity Selection, Huffman Coding, and Minimum Spanning Tree (Kruskal's/Prim's) algorithms.
- A fast, approximate, or 'good enough' answer is acceptable and computational efficiency is prioritized over guaranteed exact optimality (in problems where greedy is not provably optimal).

6.2 Dynamic Programming is Necessary When:

- Items or decisions are indivisible / binary (0/1), as in this problem, server or memory allocation, or investment decisions with fixed-size, non-splittable projects.

- The problem exhibits overlapping subproblems and optimal substructure but does NOT satisfy the greedy-choice property, meaning a locally best choice at each step can prevent reaching the true global optimum.
- A guaranteed exact optimal solution is required, and the input size ($n \times \text{capacity}$) is small enough for the pseudo-polynomial $O(n \times W)$ time/space cost to be practical.

7. Implementation (Python)

The following program implements and compares both approaches on the given dataset:

```
def knapsack_01(capacity, weights, values):
    n = len(weights)
    dp = [[0] * (capacity + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for w in range(capacity + 1):
            if weights[i - 1] <= w:
                dp[i][w] = max(dp[i - 1][w],
                               values[i - 1] + dp[i - 1][w - weights[i - 1]])
            else:
                dp[i][w] = dp[i - 1][w]
    # Backtrack to find selected items
    selected, w = [], capacity
    for i in range(n, 0, -1):
        if dp[i][w] != dp[i - 1][w]:
            selected.append(i - 1)
            w -= weights[i - 1]
    return dp[n][capacity], sorted(selected)

def knapsack_fractional(capacity, weights, values):
    n = len(weights)
    items = sorted(range(n), key=lambda i: values[i] / weights[i],
reverse=True)
    remaining, total_value, breakdown = capacity, 0.0, []
    for i in items:
        if remaining <= 0:
            break
        if weights[i] <= remaining:
            total_value += values[i]
            breakdown.append((i, 1.0))
            remaining -= weights[i]
        else:
            fraction = remaining / weights[i]
            total_value += values[i] * fraction
            breakdown.append((i, fraction))
            remaining = 0
    return total_value, breakdown

if __name__ == '__main__':
    weights = [10, 20, 30]
    values = [60, 100, 120]
    capacity = 50
```

```

    dp_value, dp_items = knapsack_01(capacity, weights, values)
    print(f'0/1 Knapsack (DP)           : value = {dp_value}, items = {dp_items}')

    greedy_value, greedy_items = knapsack_fractional(capacity, weights, values)
    print(f'Fractional Knapsack (Greedy): value = {greedy_value}, breakdown = {greedy_items}')

```

7.1 Program Output

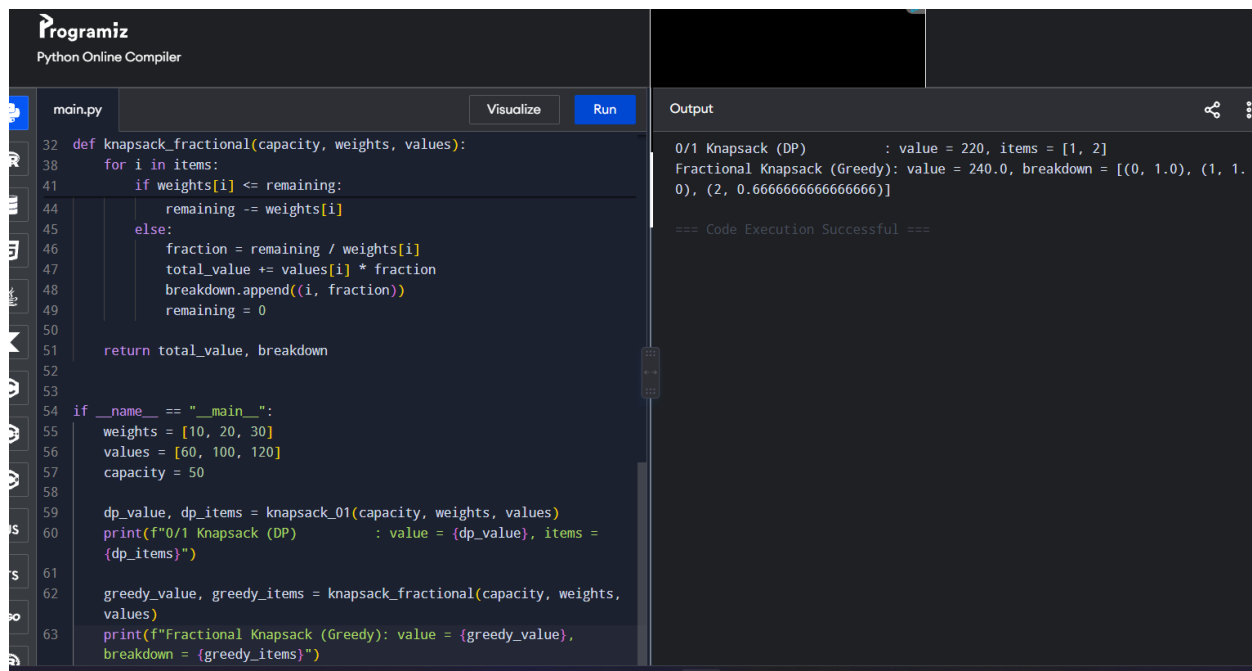
```

0/1 Knapsack (DP)           : value = 220, items = [1, 2]
Fractional Knapsack (Greedy): value = 240.0, breakdown = [(0, 1.0), (1, 1.0), (2, 0.667)]

```

(Item indices are 0-based: index 0 = Item1, index 1 = Item2, index 2 = Item3. This output matches the manual calculations in Sections 2 and 3.)

OUTPUT:



The screenshot shows the Programiz Python Online Compiler interface. On the left, the code editor displays the implementation of the knapsack algorithms. On the right, the output panel shows the results of the code execution.

Code Editor (main.py):

```

32 def knapsack_fractional(capacity, weights, values):
33     for i in items:
34         if weights[i] <= remaining:
35             remaining -= weights[i]
36         else:
37             fraction = remaining / weights[i]
38             total_value += values[i] * fraction
39             breakdown.append((i, fraction))
40             remaining = 0
41     return total_value, breakdown
42
43 if __name__ == "__main__":
44     weights = [10, 20, 30]
45     values = [60, 100, 120]
46     capacity = 50
47
48     dp_value, dp_items = knapsack_01(capacity, weights, values)
49     print(f'0/1 Knapsack (DP)           : value = {dp_value}, items = {dp_items}')
50
51     greedy_value, greedy_items = knapsack_fractional(capacity, weights, values)
52     print(f'Fractional Knapsack (Greedy): value = {greedy_value}, breakdown = {greedy_items}')

```

Output Panel:

```

0/1 Knapsack (DP)           : value = 220, items = [1, 2]
Fractional Knapsack (Greedy): value = 240.0, breakdown = [(0, 1.0), (1, 1.0), (2, 0.6666666666666666)]
=== Code Execution Successful ===

```

8. Suggested Tools and Platforms for Implementation

- Programming language: Python 3 (or Java/C++) for implementing and testing both algorithms.
- IDE / Editor: Visual Studio Code or PyCharm for development and debugging.
- Version control & submission: Git and GitHub, for hosting the Source_Code and Report as required by the submission instructions.
- Testing: Manual test cases / Python's unittest module to validate both functions against multiple weight-value-capacity combinations.
- Visualization (optional): Matplotlib, to plot value obtained vs. capacity for both methods, highlighting the optimality gap.

9. Performance Analysis – Time and Space Complexity

Approach	Time Complexity	Space Complexity
0/1 Knapsack – Dynamic Programming	$O(n \times W)$	$O(n \times W)$ ($O(W)$ if space-optimized)
Fractional Knapsack – Greedy	$O(n \log n)$	$O(n)$ (or $O(1)$ extra, excluding input)

Here n is the number of items (3 in this instance) and W is the knapsack capacity (50). The Greedy Fractional Knapsack approach is asymptotically far more efficient, dominated purely by the cost of sorting items by ratio ($O(n \log n)$), and does not depend on the capacity value at all. The DP-based 0/1 Knapsack approach depends on both n and W (pseudo-polynomial time), which is more expensive but necessary to guarantee correctness under the indivisibility constraint.

10. Insights on Method Selection

The comparison shows that algorithm choice should be driven by the nature of the constraint, not merely by which method returns a higher numeric value. The Fractional Knapsack Greedy method produced a higher value (240) purely because it was allowed to split Item 3 – an option that does not exist in genuinely indivisible real-world scenarios. If the underlying resource (e.g., a server, a physical unit, a whole project) truly cannot be split, using Greedy on the assumption that a whole-item version will yield a similarly optimal or near-optimal answer is unsafe, as Section 4 demonstrates a real 20-value gap between the two solutions for this exact dataset.

The practical decision rule is: use Greedy when resources or items are divisible, when the greedy-choice property has been proven for the given problem, and when speed matters more than the cost of implementing/maintaining a DP table. Use Dynamic Programming when items are indivisible (0/1), when correctness must be guaranteed for a bounded and moderate-sized input, and when the problem has overlapping subproblems and optimal substructure without a valid greedy-choice property. Hybrid strategies are also common in industry: Greedy or heuristic methods are used for quick approximate solutions or upper-bound estimates (as Section 4 shows, the Greedy fractional value is a valid upper bound on the 0/1 optimum), while DP or exact solvers are reserved for final, high-stakes allocation decisions.

11. Conclusion

For the given dataset (weights [10, 20, 30], values [60, 100, 120], capacity 50), the 0/1 Knapsack solved via Dynamic Programming yields an optimal value of 220 by selecting Item 2 and Item 3 in full, while the Fractional Knapsack solved via the Greedy strategy yields a higher value of 240 by taking Item 1 and Item 2 fully and 66.7% of Item 3. Both results are correct and optimal for their respective problem formulations. The key insight is that Greedy is optimal only when fractional selection is valid, whereas Dynamic Programming is required to guarantee optimality under the stricter, and often more realistic, 0/1 (indivisible) constraint. Method selection should therefore be based on the divisibility of the underlying resource and whether the problem satisfies the greedy-choice property, not on which algorithm is simpler or faster to run.